Memoria_Grupo7_PLN

Grupo 7

2024-12-14

Información Integrantes

Número de grupo: 7 Emails de los integrantes y tareas: - abdelaziz.aarourou@alumnos.upm.es - daniel.barahonamor@alumnos.upm.es - jl.garcia.arocas@alumnos.upm.es - lucia.mmiramontes@alumnos.upm.es - pablo.tsang@alumnos.upm.es

Introducción

La práctica en cuestión tiene como objetivo desarrollar un crawler básico para una página web publicadora de artículos científicos llamada The Conversation (THC). Este funcionará de forma que se recolecten el texto de los artículos en documentos con sus respectivas urls como docvars además de otra información relevante del artículo, para esto se tendrá que escribir código que permita extraer limpiar los artículos, que permita obtener una lista de 4 urls correspondientes a artículos relacionados los cuales también se incluirán en el corpus, y por supuesto el crawler en sí.

Finalmente, con los datos recopilados se podrán realizar análisis para responder preguntas sobre estadísticas de publicación y contenido, empleando técnicas de análisis textual y visualización de datos.

*Nota para el lector: No es necesario ejecutar los apartados marcados como "Código fuente".

Pregunta 1

Enunciado: Haz un programa thc_extract.R que lea un fichero .html y genere el texto del artículo. Pablo Tsang Chang: 80% Lucia Martinez Miramontes: 10% José García Arocas: 10%

Código fuente:

```
if (!requireNamespace("rvest", quietly = TRUE)) install.packages("rvest")
if (!requireNamespace("httr", quietly = TRUE)) install.packages("httr")

library(rvest)
library(httr)

# función de obtención de contenido del HTML y limpieza de este
limpiar_html <- function(lineas){

  contenido <- grep('.*<p>.*</p>',lineas, perl = TRUE, value = TRUE)
  solo_parrafos <- contenido[-grep('.*<p><.*>.*</.*></p>', contenido, perl = TRUE)]
  parrafos_sin_ws <- trimws(solo_parrafos, which = "both", whitespace = "[ \t\r\n]")
  sin_marks_parrafo <- gsub('<p>(.*)</p>', '\\1', parrafos_sin_ws)
  patron_etiquetas <- '<(/?a.*?|/?em|/?li|/?strong|/?div.*?)>'
  sin_etiquetas <- sapply(sin_marks_parrafo, function(x) gsub(patron_etiquetas, '', x))
  contenido_limpio <- paste(sin_etiquetas, collapse = "\n")
```

```
  return(contenido_limpio)
}

# función principal, obtención del HTML a partir del url y retorna en artículo limpio mediante limpiar_
extraer_articulo <- function(url){
  contenido <- tryCatch({
    readLines(url, warn = FALSE)
  }, error = function(e) {
    message(paste("Error al descargar la URL:", url))
    return(NULL)
  })

  if (is.null(contenido)) {
    message("Error al extraer el contenido de la url.")
    return(NULL)
  }

  texto <- limpiar_html(contenido)

  if (nchar(texto) == 0) {
    message("No se ha extraído ningún texto.")
    return(NULL)
  }

  return(texto)
}
```

Explicación y Dificultades: El objetivo del archivo thc_extract.R es extraer a partir de un fichero html correspondiente a un artículo de THC el texto de dicho artículo, además de limpiar este mediante la eliminación de elementos de marcado HTML, generando de esta forma el texto del artículo únicamente. Más concretamente el texto a extraer es aquel contenido entre

y

a partir de la linea con la sentencia itemprop=“articleBody”.

Para esto se ha definido una función limpiar_html que realiza tanto la extracción del texto del archivo html como la limpieza de este, y para completar el archivo R se añade una función que extrae el texto directamente de la url correspondiente al archivo de la página THC y que utiliza la función anterior para obtener el texto final, con el objetivo de realizar pruebas.

La función limpiar_html recibe como parámetro de entrada la cadena de texto que consiste en el contenido HTML extraído de la página web, primeramente se limita a extraer todos los párrafos de dicho HTML, es decir, todo texto que se ecuentra entre las etiquetas

y

, con la expresión regular “.*

.*

” y permitiendo las expresiones regulares Perl, seguidamente se elimina del vector contenido resultante aquellos párrafos que contienen únicamente otras etiquetas HTML como “

. . .

“, de esta manera eliminamos todo párrafo que no contenga texto del artículo. Se eliminan los espacios en blanco con trimws incluyendo tabulaciones y saltos de línea. Se eliminan ya del texto las etiquetas HTML, empezando por las de párrafos y luego las etiquetas de tipo , ,

, y

, además en la expresión regular se usa /? para que se capturen tanto las etiquetas de apertura como las de cerrar. Esto se hace mediante sapply que aplica el gsub a cada párrafo. Ya finalmente con el vector de párrafos llamado sin_etiquetas unimos todos sus elementos usando el salto de línea como unión para generar lo que sería el texto del artículo completamente limpio y devolverlo.

Finalmente en la fución extraer_artículo como se ha explicado antes simplemente se extrae lo que sería la cadena de texto HTML de la url con readLines y usando tryCatch para poder manejar casos de error como errores de network o urls inválidas, devolviendo un mensaje y retornando NULL en la función, con este contenido extraído simplemente se aplica la función limpiar_html para devolver el texto limpio y se añade el manejo de ciertos casos como el de que el contenido extraído del url sea vacío o que el texto limpio de ese sea también vacío, en ambos casos retornando NULL.

Es importante notar que al saber que se estará trabajando con artículos todos provenientes de la misma plataforma asumimos una consistencia en el formato de los HTMLs a manejar lo que nos ha permitido simplificar el proceso de obtención del texto.

Pruebas y Resultados:

```
echo = TRUE
url_art <- "https://theconversation.com/los-dumbphones-o-telefonos-tontos-una-ayuda-en-la-tendencia-de-
articulo <- extraer_articulo(url_art)
substring(articulo, 1, 500)
```

```
## [1] "Es difícil confirmar cuántos dumbphones se venden en España actualmente. Estudios realizados po
```

```
print("...")
```

```
## [1] "..."
```

```
substring(articulo, nchar(articulo)-500, nchar(articulo))
```

```
## [1] " ayudar en la gestión de los problemas que hemos apuntado, no se debe recurrir a ella en exclus
```

Pregunta 2

Enunciado: Extiende el programa anterior para que también se muestren las URLs de los artículos relacionados.

Pablo Tsang Chang: 33% Lucia Martinez Miramontes: 33% José García Arocas: 33%

Código fuente (archivo):

```
if (!requireNamespace("rvest", quietly = TRUE)) install.packages("rvest")
if (!requireNamespace("httr", quietly = TRUE)) install.packages("httr")
if (!requireNamespace("reticulate", quietly = TRUE)) install.packages("reticulate")

library(rvest)
library(httr)
library(reticulate)

# función de obtención de contenido del HTML y limpieza de este
limpiar_html <- function(lineas){

  contenido <- grep('.*<p>.*</p>',lineas, perl = TRUE, value = TRUE)
  solo_parrafos <- contenido[-grep('.*<p><.*>.*</.*></p>', contenido, perl = TRUE)]
  parrafos_sin_ws <- trimws(solo_parrafos, which = "both", whitespace = "[ \t\r\n]")
  sin_marks_parrafo <- gsub('<p>(.*)</p>', '\\1', parrafos_sin_ws)
  patron_etiquetas <- '<(/?a.*?|/?em|/?li|/?strong|/?div.*?)>'
```

```
  sin_etiquetas <- sapply(sin_marks_parrafo, function(x) gsub(patron_etiquetas, '', x))
  contenido_limpio <- paste(sin_etiquetas, collapse = "\n")

  return(contenido_limpio)
}

source_python("prueba_selnium_r.py")
# función principal, obtención del HTML a partir del url y retorna en artículo limpio mediante limpiar_
extraer_articulo <- function(url){
  contenido <- tryCatch({
    readLines(url, warn = FALSE)
  }, error = function(e) {
    message(paste("Error al descargar la URL:", url))
    return(NULL)
  })

  if (is.null(contenido)) {
    message("Error al extraer el contenido de la url.")
    return(NULL)
  }

  texto <- limpiar_html(contenido)

  if (nchar(texto) == 0) {
    message("No se ha extraído ningún texto.")
    return(NULL)
  }

  # obtener lista de urls relacionadas
  urls_rel <- get_urls_rel(url)

  return(list(texto = texto, urls_rel = urls_rel))
}
```

Código fuente (archivo python para scrapping):

```
#En el caso de que no se tenga selenium instalado
import subprocess
import sys

try:
    import selenium
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "selenium"])
    import selenium

import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.keys import Keys
from typing import List, Optional
```

```python
def get_urls_rel(articulo: str, timeout: int = 60) -> Optional[List[str]]:
    start_time = time.time()  # Marca el inicio de la ejecución
    driver = webdriver.Chrome()

    try:
        driver.get(articulo)

        # Aceptación de cookies
        try:
            if time.time() - start_time > timeout:
                raise TimeoutError("Tiempo límite excedido.")
            WebDriverWait(driver, 10).until(
                EC.element_to_be_clickable((By.XPATH, '//*[@id="didomi-notice-agree-button"]'))
            ).click()
        except Exception:
            return None

        # Manejo de pop-ups
        try:
            if time.time() - start_time > timeout:
                raise TimeoutError("Tiempo límite excedido.")
            popup = WebDriverWait(driver, timeout=2).until(
                EC.element_to_be_clickable((By.XPATH, '//*[@id="__next"]/div[2]/div/div[2]/div/div/div[1
            )
            popup.click()
        except Exception:
            pass

        try:
            if time.time() - start_time > timeout:
                raise TimeoutError("Tiempo límite excedido.")
            popup2 = WebDriverWait(driver, timeout=2).until(
                EC.element_to_be_clickable((By.XPATH, '/html/body/div[5]/div[3]/div/div[1]/button'))
            )
            popup2.click()
        except Exception:
            pass

        # Scroll y búsqueda de URLs relacionadas
        try:
            body = driver.find_element(By.TAG_NAME, "body")

            while not driver.find_elements(By.XPATH, '//*[@id="article"]/div[2]/div/div/div/div/article/
                if time.time() - start_time > timeout:
                    raise TimeoutError("Tiempo límite excedido.")
                body.send_keys(Keys.PAGE_DOWN)
                time.sleep(0.1)

            elements = WebDriverWait(driver, 10).until(
                EC.presence_of_all_elements_located(
                    (By.XPATH, '//*[@id="article"]/div[2]/div/div/div/div/article/header/h2/a')
                )
            )
```

```
            urls_relacionadas = list(set(element.get_attribute("href") for element in elements))

        except Exception as e:
            return None

    except TimeoutError as e:
        return None

    except Exception as e:
        return None

    finally:
        # Asegurarse de cerrar el navegador
        driver.quit()

    return urls_relacionadas
```

Explicación y Dificultades: En este caso se busca extender el archivo thc_extract.R anterior de manera que no solo obtenga el texto de un artículo limpio si no que, al final de cada artículo en THC hay una sección de artículos relacionados con el actual, donde se incluyen 4 artículos adicionales y se quiere devolver las urls de estos para poder extraer información de ellos en el crawler.

Para lograr este fin, en adición a la función limpiar_html se ha creado una función nueva llamada get_urls_rel y definida en un archivo python que realiza web scrapping mediante la librería Selenium a la página del artículo y extrae las 4 urls adicionales. Este archivo es intergrado en thc_extract_extendido.R mediante la librería reticulate, que implementa la función source_python("prueba_selnium_r.py"). Al ejecutarse esta sentencia se ejecuta el archivo python definido de forma que la función definida en este se guarda en el entorno de R y puede ser llamada.

Esta función toma la url del artículo principal y un valor de timeout (cuyo valor implícito es 60 segundos) para determinar cuando se considera un error y usa Chrome como driver. Mediante bloques try-except se trata de aceptar las cookies de la página automatizadamente identificando y clickeando en el botón, seguidamente hay un manejo de pop-ups, en concreto dos pop-ups específicos que podrían saltar, en este caso solo habría que identificarlos mediante el path y clickearlos para seguir.

Una vez resuelto esto se pasa a la obtención de las urls en la sección denominada "También le podría interesar", para ello se baja por la página, es decir, se simula la accion de "scroll down", identificamos el elemento body que será el principal de la página y mediante un bucle while se realiza la accion de body.send_keys(Keys.PAGE_DOWN) desde el body hasta que se identifique el elemento correspondiente a las urls de los artículos relacionados, esto lo que hace es deslizar la página hacia abajo, una vez se identifiquen dichos elementos se sale del bucle y se acumulan en la variable elements, finalmente se itera por esta obteniendo cada uno de los "elements" recogidos que deberían ser 4 y de estos se extrae el valor de su atributo href que contiene en sí la url como string, y estos se almacenan en una lista llamada urls_relacionadas que será la que retorne dicha función.

Vemos que en todas las acciones hay una gestión de errores con except tal que si saltan estos la función devuelva None y se imprima un mensaje indicando el contexto de dicho error. A mayores hay un control de timeout a lo largo de las acciones principales de la función de manera que si el tiempo actual menos el tiempo de comienzo (es decir la duración de la ejecución) has superado el límite de tiempo definido se pare la ejecución is se levante un TimeoutError.

Volviendo al archivo R, la modificación incluye una llamada a la función get_urls_rel desde la función extraer_articulo explicada en el apartado anterior. Dentro de esta se mete como parámetro de entrada la url de la cual se acaba de extraer el texto del artículo, y se guarda el valor retornado, es decir la lista de urls relacionadas, en una variable urls_rel que será retornada junto con el texto limpio en una lista, siendo el texto el primer elemento y la lista de urls el segundo.

Pruebas y Resultados:

```
echo = TRUE
url_art <- "https://theconversation.com/los-dumbphones-o-telefonos-tontos-una-ayuda-en-la-tendencia-de-l
artic_con_urls <- extraer_articulo(url_art)
substring(artic_con_urls$texto, 1, 500)
```

```
## [1] "Es difícil confirmar cuántos dumbphones se venden en España actualmente. Estudios realizados por
```

```
print("...")
```

```
## [1] "..."
```

```
substring(articulo, nchar(artic_con_urls$texto)-500, nchar(artic_con_urls$texto))
```

```
## [1] " ayudar en la gestión de los problemas que hemos apuntado, no se debe recurrir a ella en exclusi
```

```
print(artic_con_urls$urls_rel)
```

```
## [1] "https://theconversation.com/como-se-informan-los-jovenes-en-redes-sociales-aunque-tienen-poca-co
## [2] "https://theconversation.com/es-npc-un-insulto-guia-para-entenderse-entre-generaciones-223367"
## [3] "https://theconversation.com/roblox-fortnite-y-minecraft-recogen-las-protestas-sociales-y-politi
## [4] "https://theconversation.com/el-primer-telefono-no-solo-es-cuando-sino-como-244457"
```

Pregunta 3

Enunciado: Haz un programa thc_crawler.R ...

Abdelaziz Aarourou: 50% Daniel Barahona Amor: 50%

Código fuente:

```
if (!requireNamespace("rvest", quietly = TRUE)) install.packages("rvest")
if (!requireNamespace("httr", quietly = TRUE)) install.packages("httr")
if (!requireNamespace("reticulate", quietly = TRUE)) install.packages("reticulate")
if (!requireNamespace("ggplot2", quietly = TRUE)) install.packages("ggplot2")
if (!requireNamespace("quanteda", quietly = TRUE)) install.packages("quanteda")
```

```
## Warning in .recacheSubclasses(def@className, def, env): undefined subclass
## "ndiMatrix" of class "replValueSp"; definition not updated
```

```
library(rvest)
library(httr)
library(quanteda)
```

```
## Warning: package 'quanteda' was built under R version 4.3.3
```

```
## Package version: 4.1.0
## Unicode version: 15.1
## ICU version: 74.1
```

```
## Parallel computing: 16 of 16 threads used.
```

```
## See https://quanteda.io for tutorials and examples.
```

```
library(reticulate)
library(ggplot2)

limpiar_html <- function(lineas){

  contenido <- grep('.*<p>.*</p>',lineas, perl = TRUE, value = TRUE)
```

```r
  solo_parrafos <- contenido[-grep('.*<p><.*>.*</.*></p>', contenido, perl = TRUE)]
  parrafos_sin_ws <- trimws(solo_parrafos, which = "both", whitespace = "[ \t\r\n]")
  sin_marks_parrafo <- gsub('<p>(.*)</p>', '\\1', parrafos_sin_ws)
  patron_etiquetas <- '<(/?a.*?|/?em|/?li|/?strong|/?div.*?)>'
  sin_etiquetas <- sapply(sin_marks_parrafo, function(x) gsub(patron_etiquetas, '', x))
  contenido_limpio <- paste(sin_etiquetas, collapse = "\n")

  return(contenido_limpio)
}

source_python("prueba_selnium_r.py")
thc_crawler <- function(url_articulo, tiempo_espera, duracion) {
  # Llamar al crawler de Python para obtener las URLs relacionadas
  urls <- get_urls_rel(url_articulo)

  if (is.null(urls)) {
    stop("No se pudieron obtener URLs relacionadas.")
  }

  corpus <- list()  # Para almacenar los artículos descargados
  visited_urls <- c()  # Para rastrear las URLs que ya hemos visitado

  # Mientras no hayamos llegado al tiempo impuesto
  inicio <- Sys.time()
  while(as.numeric(difftime(Sys.time(), inicio, units = "secs")) < duracion) {
    # Elegir una URL relacionada de la lista (solo si no se ha visitado antes)
    url <- urls[1]
    urls <- urls[-1]  # Eliminar la URL seleccionada de la lista de URLs a explorar
    if (url %in% visited_urls) {
      next  # Si ya hemos visitado esta URL, saltamos al siguiente
    }

    # Agregar la URL a la lista de URLs visitadas
    visited_urls <- c(visited_urls, url)

    # Descargar contenido
    contenido <- tryCatch({
      readLines(url, warn = FALSE)
    }, error = function(e) {
      message(paste("Error al descargar la URL:", url))
      return(NULL)
    })

    if (is.null(contenido)) next

    # Limpiar la información
    texto <- limpiar_html(contenido)

    # Guardar en el corpus
    documento <- list(
      texto = texto,
      url = url
    )
```

```
    corpus[[length(corpus) + 1]] <- documento

    # Obtener más URLs relacionadas del artículo actual
    urls_relacionadas <- get_urls_rel(url)

    # Añadir estas nuevas URLs a la lista de URLs a explorar
    if (!is.null(urls_relacionadas)) {
      # Filtrar URLs para que no estén ya en la lista
      urls <- unique(c(urls, setdiff(urls_relacionadas, visited_urls)))
    }

    Sys.sleep(tiempo_espera)  # Esperar antes de cada nueva petición
  }

  # Crear un corpus de quanteda con los textos descargados
  docs <- sapply(corpus, function(doc) doc$texto)
  metadata <- data.frame(
    url = sapply(corpus, function(doc) ifelse(is.null(doc$url), "", doc$url)),
    stringsAsFactors = FALSE
  )

  quanteda_corpus <- corpus(docs, docvars = metadata)
  return(quanteda_corpus)
}
```

Explicación y Dificultades:

El objetivo del archivo **crawler.R** es implementar un crawler básico, recopilando el contenido de artículos y sus URLs relacionándolos mediante un corpus y sus docvars. Este programa está diseñado para explorar un número de artículos en función de un tiempo de ejecución predefinido. Se parte de una URL inicial de un artículo, y mediante la función get_urls_rel, se extraen las URLs de los artículos relacionados para explorarlos recursivamente.

El proceso del crawler comienza estableciendo un límite de tiempo total de ejecución (duracion) y un intervalo de espera entre iteraciones (tiempo_espera) para evitar sobrecargar el servidor. Durante cada iteración, se selecciona una URL no visitada de la lista de relacionadas, se descarga su contenido mediante readLines, y se limpia el texto utilizando la función limpiar_html. Este contenido se almacena en el corpus, junto con su URL como docvar.

Además se añaden varios manejos de errores. Por ejemplo, si la función get_urls_rel no retorna resultados, el programa finaliza con un mensaje de error. Además, si readLines no puede descargar el contenido de una URL, se guarda la excepción y se registra el fallo pero no se interrumpe el proceso principal.

Para mejorar la eficiencia del tiempo y las URLs se lleva un registro de las URLs ya visitadas para evitar duplicaciones, y se asegura que el crawler termine en el tiempo definido.

Pruebas y Resultados:

```
resultado <- thc_crawler("https://theconversation.com/por-que-tergiversamos-la-informacion-cuando-amena
head(resultado)
```

```
## Corpus consisting of 6 documents and 1 docvar.
## text1 :
## "Escribo este artículo días después de que la dana llegara a ..."
##
## text2 :
## "El martes 29 de octubre de 2024, entre las 7:31 y las 7:36, ..."
```

```
##
## text3 :
## "Aunque los complots imaginarios han sido una constante histó..."
##
## text4 :
## "La tensa visita de Felipe VI y el presidente Pedro Sánchez a..."
##
## text5 :
## "Lo ocurrido el martes pasado en Valencia, desgraciadamente r..."
##
## text6 :
## "Es indescriptible pensar que, en tan solo unos minutos, pued..."
```

```
docvarss <- docvars(resultado)
```

Pregunta 4

Enunciado: Modifica el programa anterior para incluir esta información como docvars. Abdelaziz Aarourou: 50% Daniel Barahona Amor: 50%

Código fuente:

```
if (!requireNamespace("rvest", quietly = TRUE)) install.packages("rvest")
if (!requireNamespace("httr", quietly = TRUE)) install.packages("httr")
if (!requireNamespace("reticulate", quietly = TRUE)) install.packages("reticulate")
if (!requireNamespace("ggplot2", quietly = TRUE)) install.packages("ggplot2")
if (!requireNamespace("quanteda", quietly = TRUE)) install.packages("quanteda")

library(rvest)
library(httr)
library(quanteda)
library(reticulate)
library(ggplot2)

limpiar_html <- function(lineas){

  contenido <- grep('.*<p>.*</p>',lineas, perl = TRUE, value = TRUE)
  solo_parrafos <- contenido[-grep('.*<p><.*>.*</.*></p>', contenido, perl = TRUE)]
  parrafos_sin_ws <- trimws(solo_parrafos, which = "both", whitespace = "[ \t\r\n]")
  sin_marks_parrafo <- gsub('<p>(.*)</p>', '\\1', parrafos_sin_ws)
  patron_etiquetas <- '<(/?a.*?|/?em|/?li|/?strong|/?div.*?)>'
  sin_etiquetas <- sapply(sin_marks_parrafo, function(x) gsub(patron_etiquetas, '', x))
  contenido_limpio <- paste(sin_etiquetas, collapse = "\n")

  return(contenido_limpio)
}
sacar_publicacion <- function(lineas){

  patron_publicacion <- '<time datetime=.*>Publicado: (.*)</time>'
  publicacion_sucio <- grep(patron_publicacion, lineas, perl = TRUE, value = TRUE)
  publicacion_limpio <- gsub(patron_publicacion,'\\1', publicacion_sucio)
  publicacion_sin_ws <- trimws(publicacion_limpio, which = "both", whitespace = " ")

  return(publicacion_sin_ws)
}
```

```r
sacar_autoria <- function(lineas){

  patron_autor <- '<meta name="author" content="(.*)" />'
  autor_sucio <- grep(patron_autor, lineas, perl = TRUE, value = TRUE)
  autor_limpio <- gsub(patron_autor, '\\1', autor_sucio)
  autor_sin_ws <- trimws(autor_limpio, which = "both", whitespace = " ")

  return(autor_sin_ws)
}

sacar_institucion <- function(lineas){

  patron_institucion <- '.*<em class="institution_name">(.*)</em>'
  institucion_sucio <- grep(patron_institucion, lineas, perl = TRUE, value = TRUE)
  institucion_limpio <- gsub(patron_institucion, '\\1', institucion_sucio)
  institucion_sin_ws <- trimws(institucion_limpio, which = "both", whitespace = " ")

  return(institucion_sin_ws)
}

sacar_descripcion <- function(lineas){

  patron_descripcion <- '<meta property="og:description" content=(.*)/>'
  descripcion_sucio <- grep(patron_descripcion, lineas, perl = TRUE, value = TRUE)
  descripcion_limpio <- gsub(patron_descripcion, '\\1', descripcion_sucio)
  descripcion_limpio <- substring(descripcion_limpio, 2, nchar(descripcion_limpio)-2)
  descripcion_sin_ws <- trimws(descripcion_limpio, which = "both", whitespace = " ")

  return(descripcion_sin_ws)
}

sacar_keywords <- function(lineas){

  patron_keywords <- '<meta name="news_keywords" content="(.*?)" />'
  keywords_sucio <- grep(patron_keywords, lineas, perl = TRUE, value = TRUE)
  keywords_limpio <- gsub(patron_keywords, '\\1', keywords_sucio)
  keywords_sin_ws <- trimws(keywords_limpio, which = "both", whitespace = " ")

  return(keywords_sin_ws)
}
source_python("prueba_selnium_r.py")
thc_crawler <- function(url_articulo, tiempo_espera, duracion) {
  # Llamar al crawler de Python para obtener las URLs relacionadas
  urls <- get_urls_rel(url_articulo)

  if (is.null(urls)) {
    stop("No se pudieron obtener URLs relacionadas.")
  }

  corpus <- list()  # Para almacenar los artículos descargados
  visited_urls <- c()  # Para rastrear las URLs que ya hemos visitado

  # Mientras no hayamos llegado al tiempo impuesto
```

```r
inicio <- Sys.time()
while(as.numeric(difftime(Sys.time(), inicio, units = "secs")) < duracion) {
  # Elegir una URL relacionada de la lista (solo si no se ha visitado antes)
  url <- urls[1]
  urls <- urls[-1]  # Eliminar la URL seleccionada de la lista de URLs a explorar
  if (url %in% visited_urls) {
    next  # Si ya hemos visitado esta URL, saltamos al siguiente
  }

  # Agregar la URL a la lista de URLs visitadas
  visited_urls <- c(visited_urls, url)

  # Descargar contenido
  contenido <- tryCatch({
    readLines(url, warn = FALSE)
  }, error = function(e) {
    message(paste("Error al descargar la URL:", url))
    return(NULL)
  })

  if (is.null(contenido)) next

  # Limpiar y extraer la información
  texto <- limpiar_html(contenido)
  publicacion <- sacar_publicacion(contenido)
  autoria <- sacar_autoria(contenido)
  institucion <- sacar_institucion(contenido)
  descripcion <- sacar_descripcion(contenido)
  keywords <- sacar_keywords(contenido)

  # Guardar en el corpus
  documento <- list(
    texto = texto,
    publicacion = publicacion,
    autoria = autoria,
    institucion = institucion,
    descripcion = descripcion,
    keywords = keywords,
    url = url
  )
  corpus[[length(corpus) + 1]] <- documento

  # Obtener más URLs relacionadas del artículo actual
  urls_relacionadas <- get_urls_rel(url)

  # Añadir estas nuevas URLs a la lista de URLs a explorar
  if (!is.null(urls_relacionadas)) {
    # Filtrar URLs para que no estén ya en la lista
    urls <- unique(c(urls, setdiff(urls_relacionadas, visited_urls)))
  }

  Sys.sleep(tiempo_espera)  # Esperar antes de cada nueva petición
}
```

```r
  # Crear un corpus de quanteda con los textos descargados
  docs <- sapply(corpus, function(doc) doc$texto)
  metadata <- data.frame(
    publicacion = sapply(corpus, function(doc) ifelse(is.null(doc$publicacion), "", doc$publicacion)),
    autoria = sapply(corpus, function(doc) ifelse(is.null(doc$autoria), "", doc$autoria)),
    institucion = sapply(corpus, function(doc) ifelse(is.null(doc$institucion), "", doc$institucion)),
    descripcion = sapply(corpus, function(doc) ifelse(is.null(doc$descripcion), "", doc$descripcion)),
    keywords = sapply(corpus, function(doc) ifelse(is.null(doc$keywords), "", doc$keywords)),
    url = sapply(corpus, function(doc) ifelse(is.null(doc$url), "", doc$url)),
    stringsAsFactors = FALSE
  )

  quanteda_corpus <- corpus(docs, docvars = metadata)
  return(quanteda_corpus)
}

resultado <- thc_crawler("https://theconversation.com/por-que-tergiversamos-la-informacion-cuando-amena
print(resultado)
docvarss <- docvars(resultado)

#Realizamos una ordenación para presentar los autores más frecuentes
autores_mas_frecuentes<- sort(table(docvarss$autoria), decreasing = TRUE)[1:10]

#Realizamos una ordenación para presentar cuáles son las instituciones con más publicaciones
instituciones_mas_frecuentes <- sort(table(docvarss$institucion), decreasing = TRUE)[1:10]

#Calculamos el número de artículos
numero_articulos <- length(resultado)

#Convertimos las keywords en tokens, eliminamos las stopwords para quedarnos sólo
#con las palabras esenciales, las convertimos en minúsculas y posteriormente en
#un dataframe para realizar la nube de palabras
tokens <- tokens(docvarss$keywords, remove_punct = TRUE)
sin_stopwords <- tokens_remove(tokens, pattern = stopwords("es"))
token_minuscula <- tolower(sin_stopwords)
dfm <- dfm(token_minuscula)
#Asignamos una semilla para evitar la aletoriedad
set.seed(123)
nube_keywords <- quanteda.textplots::textplot_wordcloud(dfm)

#asignamos una lista para su posterior uso a la hora de modificar las fechas
meses <- c("enero" = "01", "febrero" = "02", "marzo" = "03", "abril" = "04",
           "mayo" = "05", "junio" = "06", "julio" = "07", "agosto" = "08",
           "septiembre" = "09", "octubre" = "10", "noviembre" = "11", "diciembre" = "12")

publicacion_cambiado <- docvarss$publicacion

#cambiar el nombre de los meses por su correspondiente valor numérico
for (mes in names(meses)) {
  publicacion_cambiado <- gsub(mes, meses[mes], publicacion_cambiado, ignore.case = TRUE)
}

#Sólo nos quedamos con la fecha y eliminamos la hora
```

```r
solo_fecha <- lapply(publicacion_cambiado, function(x){
  substr(x, 1, nchar(x) - 10)
})
#Modificamos la separación acorde según el formato de las funciones empleadas
fecha_final <- gsub("\\s", "-", solo_fecha)

#Convertimos el string en una fecha,m  y posteriormente en un dataframe
fechas <- as.Date(fecha_final, format = "%d-%m-%Y")
datos <- data.frame(
  fecha = fechas
)

#creamos un rango de fechas para el índice del timeline
rango_fechas <- seq(min(fechas), max(fechas), by = "6 month")

#creación del timeline
ggplot(datos, aes(x = fechas)) +
  geom_segment(aes(xend = fechas, y = 0, yend = 1), color = "blue", size = 0.5) +
  scale_x_date(
    breaks = rango_fechas,
    date_labels = "%d-%m-%Y"
  ) +
  labs(x = "Fecha", y = NULL, title = "Línea Temporal") +
  theme_minimal() +
  theme(
    axis.text.y = element_blank(),
    axis.text.x = element_text(angle = 45, hjust = 1)
  )
```

Explicación y Dificultades:

Pruebas y Resultados:

```r
resultado <- thc_crawler("https://theconversation.com/por-que-tergiversamos-la-informacion-cuando-amena
docvarss <- docvars(resultado)
print(head(resultado))
```

```
## Corpus consisting of 6 documents and 1 docvar.
## text1 :
## "Escribo este artículo días después de que la dana llegara a ..."
##
## text2 :
## "El martes 29 de octubre de 2024, entre las 7:31 y las 7:36, ..."
##
## text3 :
## "Aunque los complots imaginarios han sido una constante histó..."
##
## text4 :
## "La tensa visita de Felipe VI y el presidente Pedro Sánchez a..."
##
## text5 :
## "Lo ocurrido el martes pasado en Valencia, desgraciadamente r..."
##
## text6 :
## "Es indescriptible pensar que, en tan solo unos minutos, pued..."
```

Cuestiones a responder

Pablo Tsang Chang: 100%

4.1. ¿Tu crawler puede escapar de "referencias circulares" (cuando en los artículos relacionados está el artículo inicial)? Si no lo hace, ¿cómo podría hacerlo?.

Sí es capaz de escapar de referencias circulares, debido a que tenemos un vector con las URLs empleadas, y en caso de que la URLs al cual se va a extraer la información se encuentre en dicho vector, se omito la URL repetida, y se salta al siguiente URL.

4.2. Deja funcionando tu crawler "toda una noche", con un tiempo entre artículos de 1 segundo, y un artículo inicial de la sección de "Ciencia + Tecnología". ¿Cuántos artículos has obtenido?. ¿Cuál es el autor con más publicaciones?. ¿Y la institución que más publica?. Haz una nube de palabras con las palabrasclave de los artículos obtenidos. Muestra una gráfica (línea temporal) con las fechas de las publicaciones.

Primero vamos a importar los resultados que hemos obtenido tras la ejecución del programa durante 4 horas (se explicará posteriormente la decisión de ejecutarlo durante 4 horas).

```
load("resultados_4h.RData")
```

El resultado que obtenemos son de 365 publicaciones

Ahora a partir de los datos obtenemos los autores más frecuentes, observando que el autor con más publicaciones es Raúl Magallón Rosa.

```
#Realizamos una ordenación para presentar los autores más frecuentes
autores_mas_frecuentes<- sort(table(docvarss$autoria), decreasing = TRUE)[1:10]
autores_mas_frecuentes
```

```
##
##           Raúl Magallón Rosa           Roberto R. Aramayo
##                            4                            4
## Antonio Ruiz de Elvira Serra          Daniel Jato Espino
##                            3                            3
##            Emilio Muñoz Ruiz     Rosario Moreno Soldevila
##                            3                            3
##      Antonio Figueras Huerta         Arturo Fuentes Calle
##                            2                            2
##                Astrid Wagner                  Carlos Galé
##                            2                            2
```

Para ver cuál es la institución con más publicaciones realizamos una operación similar, y teniendo como respuesta que la Universidad Complutense de Madrid es la que más publicaciones tiene.

```
#Realizamos una ordenación para presentar cuáles son las instituciones con más publicaciones
instituciones_mas_frecuentes <- sort(table(docvarss$institucion), decreasing = TRUE)[1:10]
instituciones_mas_frecuentes
```

```
##
##            Universidad Complutense de Madrid
##                                           17
##                      Universidad Carlos III
##                                           11
##            Universidad de Castilla-La Mancha
##                                           11
##                      Universidad de Granada
##                                           11
##                  Universidad Rey Juan Carlos
##                                           11
```

```
## UNIR - Universidad Internacional de La Rioja
##                                             9
##                         Universidad de Navarra
##                                             9
##               Universidad Pontificia Comillas
##                                             9
##                                IE University
##                                             8
##    Universidad de Las Palmas de Gran Canaria
##                                             8
```

Para hacer la nube de palabras empleamos el siguiente código:

```r
if (!requireNamespace("quanteda", quietly = TRUE)) install.packages("quanteda")
if (!requireNamespace("quanteda.textplots", quietly = TRUE)) install.packages("quanteda.textplots")

library(quanteda)
library(quanteda.textplots)
#Convertimos las keywords en tokens, eliminamos las stopwords para quedarnos sólo
#con las palabras esenciales, las convertimos en minúsculas y posteriormente en
#un dataframe para realizar la nube de palabras
tokens <- tokens(docvarss$keywords, remove_punct = TRUE)
sin_stopwords <- tokens_remove(tokens, pattern = stopwords("es"))
token_minuscula <- tolower(sin_stopwords)
dfm <- dfm(tokens(token_minuscula))
#Asignamos una semilla para evitar la aletoriedad
set.seed(123)
nube_keywords <- quanteda.textplots::textplot_wordcloud(dfm)
```



Tras convertir en tokens (sólo las palabras), eliminamos los stopwords para obtener las palabras relevantes y las pasamos a minúscula para que el formato sea homogéneo, y no considere una misma palabra como dos diferentes por el hecho del formato. Finalmente asignamos la semilla 123 para evitar la aletoriedad a la hora de generar la nube de palabras.

Para realizar el timeline desarrollado el siguiente código:

```
if (!requireNamespace("ggplot2", quietly = TRUE)) install.packages("ggplot2")

library(ggplot2)
#asignamos una lista para su posterior uso a la hora de modificar las fechas
meses <- c("enero" = "01", "febrero" = "02", "marzo" = "03", "abril" = "04",
           "mayo" = "05", "junio" = "06", "julio" = "07", "agosto" = "08",
           "septiembre" = "09", "octubre" = "10", "noviembre" = "11", "diciembre" = "12")

publicacion_cambiado <- docvarss$publicacion

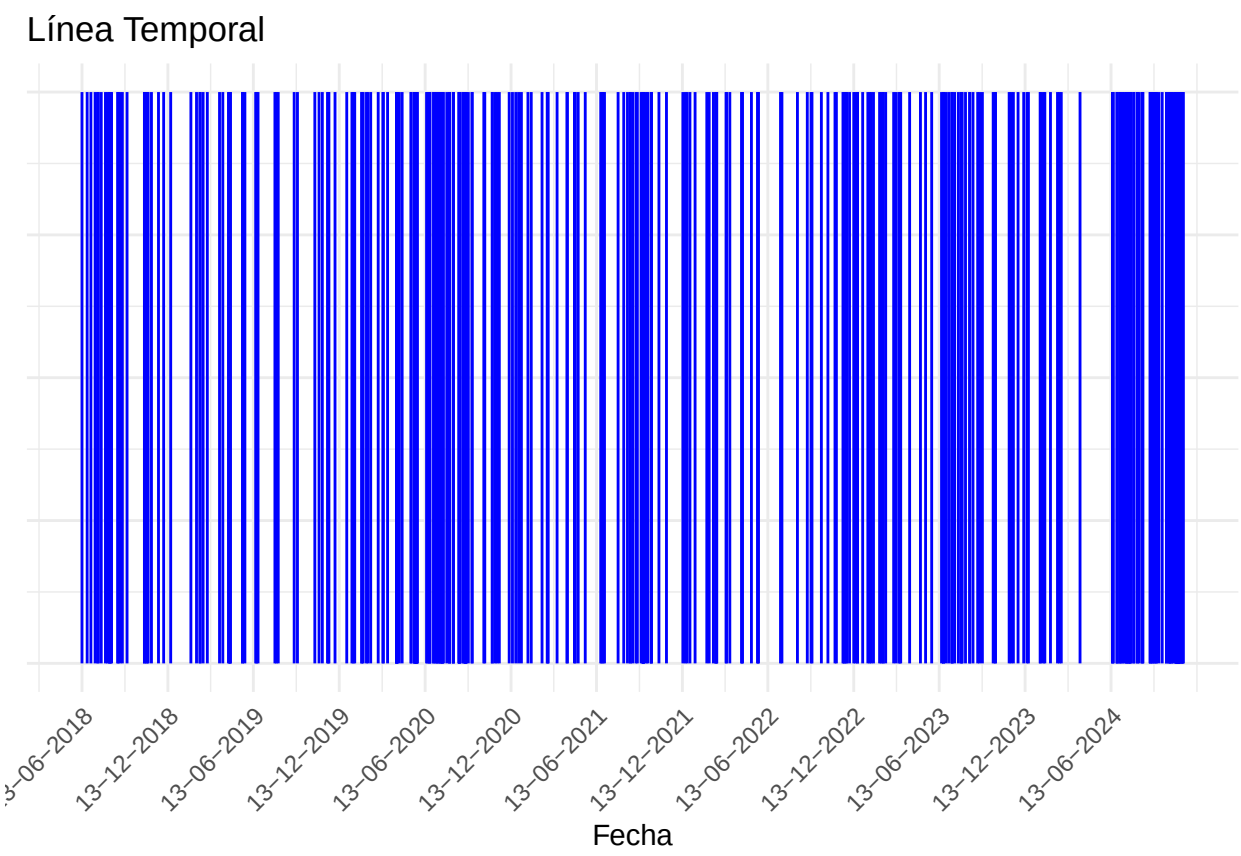
#cambiar el nombre de los meses por su correspondiente valor numérico
for (mes in names(meses)) {
  publicacion_cambiado <- gsub(mes, meses[mes], publicacion_cambiado, ignore.case = TRUE)
}

#Sólo nos quedamos con la fecha y eliminamos la hora
solo_fecha <- lapply(publicacion_cambiado, function(x){
  substr(x, 1, nchar(x) - 10)
})
#Modificamos la separación acorde según el formato de las funciones empleadas
fecha_final <- gsub("\\s", "-", solo_fecha)
```

```r
#Convertimos el string en una fecha,m  y posteriormente en un dataframe
fechas <- as.Date(fecha_final, format = "%d-%m-%Y")
datos <- data.frame(
  fecha = fechas
)

#creamos un rango de fechas para el índice del timeline
rango_fechas <- seq(min(fechas), max(fechas), by = "6 month")

#creación del timeline
ggplot(datos, aes(x = fechas)) +
  geom_segment(aes(xend = fechas, y = 0, yend = 1), color = "blue", size = 0.5) +
  scale_x_date(
    breaks = rango_fechas,
    date_labels = "%d-%m-%Y"
  ) +
  labs(x = "Fecha", y = NULL, title = "Línea Temporal") +
  theme_minimal() +
  theme(
    axis.text.y = element_blank(),
    axis.text.x = element_text(angle = 45, hjust = 1)
  )
```



Primero hemos tenido que convertir los nombres de los meses a sus correspondientes valores númericos para la correcta interpretación en as.Date, citando también algunos cambios de formato (por ejemplo los espacios por "-"). También creamos una leyenda con intervalos de fecha equiespaciados (6 meses), para una mejor

compresión de los resultados, y pintadndo la fecha de cada publicación como una línea vertical.

Dificultades: La principal dificultad que hemos enfrentado ha sido durante la ejecución del programa. A escalas pequeñas, como sesiones de 5 minutos, no se generaban ciertos pop-ups que aparecen de manera completamente aleatoria. Por esta razón, hemos tenido que monitorizar el programa durante sesiones más largas para identificar todos los pop-ups que surgían.

Además, decidimos limitar las sesiones de ejecución a 4 horas después de observar que, al extenderlas a 8 horas, aparecía un mensaje indicando un error fatal. Sospechamos que este problema está relacionado con un consumo excesivo de memoria RAM, ya que el programa maneja listas y dataframes de gran tamaño, una hipótesis respaldada por consultas realizadas en varios foros.